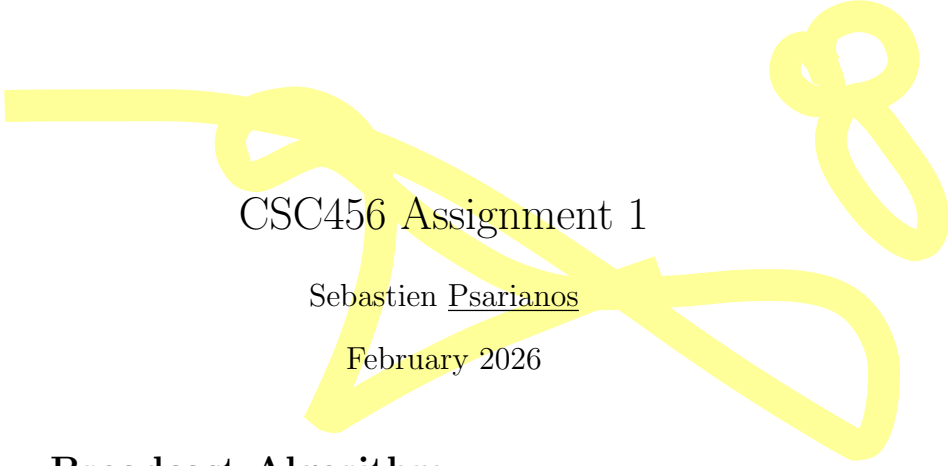




CSC456 Assignment 1

Sebastien Psarianos

February 2026

1 Broadcast Algorithm

The following code represents a one to all broadcast algorithm, designed to be used on a hypercube connected network with 2^d processors. The processor `npin` is the source processor with the original value stored in `data`. By the end of the algorithm, `data` will contain this value in all processors.

```
1 send_list is an empty list
2 for i = 0, ..., d-1
3     if bit(i, np) != bit(i, npin)
4         receive data from processor flip(i, np)
5         break
6     else
7         append flip(i,np) to send_list
8     endif
9 endfor
10
11 for i = 0, ..., length of send_list
12     send data to processor send_list[i]
13 endfor
```

`send_list` is initialized on line one to store the targets that each processor needs to send to. This is done to prevent `data` from being sent prior to it being received from the previous processor.

The algorithm will do a check for each bit value in `np` to see if it matches `npin`. While the bits match, it will append the processor with the current bit flipped to the `send_list` on line 7. Once it detects a bit that is different, the processor will then wait for the data from the corresponding processor (Line 4) and then break out of the loop, having recieved the updated `data` value.

For each bit place value, half of the processors will drop out and go into a receiving state, while half will go on to continue creating a send list. Considering that for each value in the bit representation of `np` the processor with that bit flipped is unique, there will be no duplicated send calls to any one processor.

At the end of the loop, only `npin` remains in the loop.

Once all processors are in a receive state, the next loop will be entered by `npin`, at which point, `data` will be propagated in a pairwise manner between processors. Since each processor only receives at most one piece of communication, there is no replicate communication.

For each core, the number of instructions has an upper bound of $\log(N)$ where N is the network size, and a space complexity also bound by $\log(N)$ for the `send_list` vector.

2 Global arithmetic algorithm

The goal of this section is to profile the performance of a custom global sum algorithm with the builtin `MPI_Allreduce` function provided by MPI. Additionally, performance analysis is done on a custom algorithm used to perform a series of pairwise reductions of an arbitrary length. This second algorithm is compared to a series of calls to `MPI_Allreduce` with selection of instructions. All algorithms are provided in the appendix section A.1. Both algorithms are designed to be used with a hypercube network containing 2^d processors.

2.1 Global sum up algorithm

The implementation for the global sum up algorithm is given in A.1 as the `globalSumUp` function. At the start of the algorithm it will take the value `a` in each processor and sum it with every value in every other processor. At the end of the algorithm, every processor will contain the same total sum.

It operates by pairing up processors with other processors with one flipped bit. For each processor, it will send out its stored value asynchronously, receive a new value and then perform a sum on the result, sending the updated value to the processor next in the loop. To prevent a deadlock condition, the algorithm uses the `MPI_Isend` function to allow the new data to be received while the previous data is still sending. This strategy does involve replicate communication.

2.2 Global operation series algorithm

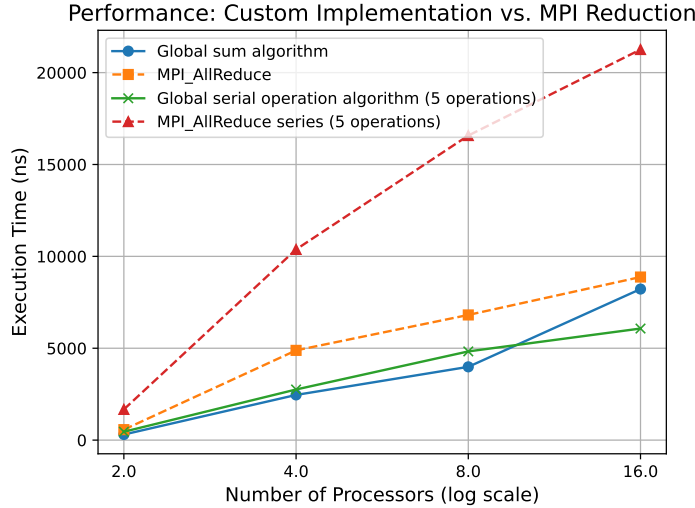
This implementation is also provided in the appendix section A.1 as the `globalInstructionSet` function. The algorithm takes in a list of instructions that are the same across all processors in the `enum {SUM, PRODUCT, MAX, MIN}`. Each processor starts with a unique value `a`. At the end of the algorithm, each processor's `vec` param will be populated with the value of applying each operation in order over the entire network.

The data propagation methodology is the same as the previous algorithm, however instead of just sending a single value for every step of communication, it sends a vector of values corresponding to the current partial value of the respective instruction.

2.3 Results

To profile the performance of these algorithms vs the built in `MPI_Allreduce` function, both functions were timed 50 times, taking the mean runtime of each function and comparing them over a range of network sizes $\{2, 4, 8, 16\}$. This performance analysis was done using the code in A.2 and plotted using `matplotlib` as shown in A.3. The results of this comparison are shown in Figure 1.

Figure 1: Performance of custom sum up and serial operation algorithms to existing MPI functionality.



Most algorithms seem to be performing with a logarithmic increase in runtime for larger processor sizes, reflecting the $O(\log p)$ communication that is required by the algorithm. the one exception is the global sum algorithm, with a large jump at the 16 processor size. However all custom algorithms performed better than the `MPI_Allreduce` algorithm counterparts. there is a very clear distinction between the runtimes of the series of `MPI_Allreduce` calls to the custom implementation with vectorized communication. This suggests that this algorithm is bound by communication time and not arithmetically bound.

3 Matrix multiplication performance analysis

The goal of this section was to profile the efficiency of a multi-processor matrix multiplication algorithm. The raw timing data is displayed in Table 1:

Table 1: Simulated performance values from assignment

p	256	384	512	1024
1	16777	56623	134218	-
2	8458	28381	67178	-
4	4333	14294	33693	-
8	2305	7286	16985	134426
16	1326	3816	8666	67386
32	871	2116	4541	33901
64	678	1301	2513	17193

3.1 Algorithm Efficiency

As shown in the course notes (Christara 2026), the relative algorithm speedup and efficiency are given by Equations 1 and 2

$$S_P = \frac{t_1}{t_P} \quad (1)$$

$$E_P = \frac{S_P}{p} \quad (2)$$

Where t_P is the execution time on P processors. By applying these formulas to the provided table, the efficiencies for the 256-Sized matrix for $P \in \{2, 8, 64\}$ are given.

$$\begin{aligned}
 S_4 &= \frac{t(1, 256)}{t(4, 256)} \implies E_4 = \frac{t(1, 256)}{t(4, 256) \cdot 4} = \frac{16777}{4333 \cdot 4} \approx 0.968 \\
 S_8 &= \frac{t(1, 256)}{t(8, 256)} \implies E_8 = \frac{t(1, 256)}{t(8, 256) \cdot 8} = \frac{16777}{2305 \cdot 8} \approx 0.910 \\
 S_{64} &= \frac{t(1, 256)}{t(64, 256)} \implies E_{64} = \frac{t(1, 256)}{t(64, 256) \cdot 64} = \frac{16777}{678 \cdot 64} \approx 0.390
 \end{aligned}$$

This shows a sharp drop at the 64 processor's speed when considering the constant problem size.

3.2 Algorithm efficiency with scaled workload size

The scaled efficiency of an algorithm is defined by Equation 3 from the course notes (Christara 2026).

$$E_p = \frac{t_1(n_1)}{t_p(n_p)} \quad (3)$$

To determine the values n_1, n_p , Equation 4 must be used (Christara 2026).

$$\tilde{t}_1(n_p) = p\tilde{t}_1(n_1) \quad (4)$$

Where $\tilde{t}_1(n)$ represents the number of operations the algorithm requires when applied to a problem of size n . The goal is to scale corresponding to a workload based problem size. Since direct matrix multiplication algorithms grow with $O(n^3)$ with matrix size, $\tilde{t}_1(n)$ can be approximated with n^3 . Using Equation 4 to solve for n' corresponding to p processors:

$$\begin{aligned} \tilde{t}_1(n_p) = p\tilde{t}_1(n_1) &\implies n_p^3 \approx p \cdot n_1^3 \\ &\implies n_p \approx \sqrt[3]{p} \cdot n_1 \end{aligned}$$

Therefore, for p processors, the matrix size should grow with $\sqrt[3]{p}$ relative to a single processor. For $n_1 = 256$, this gives: $p_4 \approx 256 \cdot \sqrt[3]{4} \approx 406$, $p_8 \approx 256 \cdot \sqrt[3]{8} = 512$ and $p_{64} = 256 \cdot \sqrt[3]{64} = 1024$. The most apt choices for analysis using the available data are therefore 384, 512 and 1024 for 4, 8 and 64 processors respectively. Using the provided values in the table, Equation 3 can be used to calculate the relative scaled efficiencies.

$$\begin{aligned} E_4 &= \frac{t(1, 256)}{t(4, 384)} = \frac{16777}{14294} \approx 1.17 \\ E_8 &= \frac{t(1, 256)}{t(8, 512)} = \frac{16777}{16985} \approx 0.980 \\ E_{64} &= \frac{t(1, 256)}{t(64, 1024)} = \frac{16777}{17193} \approx 0.976 \end{aligned}$$

The results showed an improvement in scaled efficiency for four processors relative to one, however due to the lack of data on a matrix of size ≈ 406 , it is possible that this is due to the reduced matrix size relative to the ideal scaled size. Relatively maintained efficiency was seen for 8 processors and a drastically lower efficiency at 64 processors.

3.3 Algorithm efficiency with scaled memory requirements

Using Equation 4, it is possible to determine the values n_p to determine matrix sizes for scaled memory efficiencies. The memory requirements for storing the matrixes scale with $O(n^2)$. Therefore, the problem complexity $\tilde{t}_1$ can be approximated by n^2 , giving:

$$\begin{aligned} \tilde{t}_1(n_p) = p \cdot \tilde{t}_1(n_1) &\implies n_p^2 \approx p \cdot n_1^2 \\ &\implies n_p \approx \sqrt{p} \cdot n_1 \end{aligned}$$

Therefore, for p processors, the matrix size should be scaled according to $\sqrt{p}$. For the required processor counts of 4 and 16, and for $n_1 = 256$, the appropriate

memory sizes are: $n_4 \approx 256 \cdot \sqrt{4} = 512$ and $n_{16} \approx 256 \cdot \sqrt{4} = 1024$. Using Equation 3 to calculate the relative scaled efficiencies gives:

$$E_4 = \frac{t(1, 256)}{t(4, 512)} = \frac{16777}{16985} \approx 0.500$$

$$E_{16} = \frac{t(1, 256)}{t(16, 1024)} = \frac{16777}{67386} \approx 0.250$$

4 Batched vs individual communication for a vector communication problem

The aim of this section was to compare the performance of batched communication to sending values individually when distributing vector values through a ring-topology network of processors. Two algorithms were implemented to represent both approaches. For both approaches, the algorithm begins with $p = 2^d$ processors, each containing a vector of data of size n^2 . For each processor, it will pass the values stored at $i \cdot n + 1$ to the next processor in the ring and then receive the same values from the previous processor.

4.1 Individual communication algorithm

The individual algorithm, known as `passSubVectorByComponent` is presented in appendix B.1. This algorithm sends each value in the vector individually in its own call to `MPI_Isend`.

4.2 Batched communication algorithm

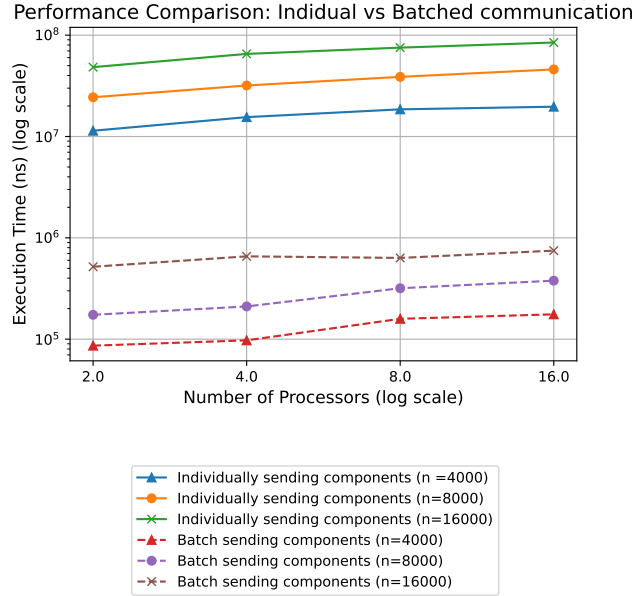
The batched algorithm, known as `passSubVectorGrouped` is also presented in appendix B.1. This algorithm dynamically allocates a send buffer that will contain all of the values that need to be transferred to the next processor and sends them in one call to `MPI_Isend`.

4.3 Results

Each algorithm was run on both a range of processor numbers $p \in \{2, 4, 8, 16\}$ and a range of array sizes $n \in \{1000, 2000, 4000, 8000, 16000\}$. Figure 2 displays a selection of these results for the $n \in \{4000, 8000, 16000\}$ array sizes. The profiling was done using the code in the appendix section B.2. A python script was made to run the profiling on multiple different network sizes and plot the results using `matplotlib`, shown in appendix section B.3.

There is a very clear improvement for the algorithms that utilized the batched sending protocol, shown by the dotted lines. The scale in Figure 2 is logarithmic on both the x and y scale, showing an improvement of on average around

Figure 2: Performance of the batched and individual communication algorithms over a range of array sizes.

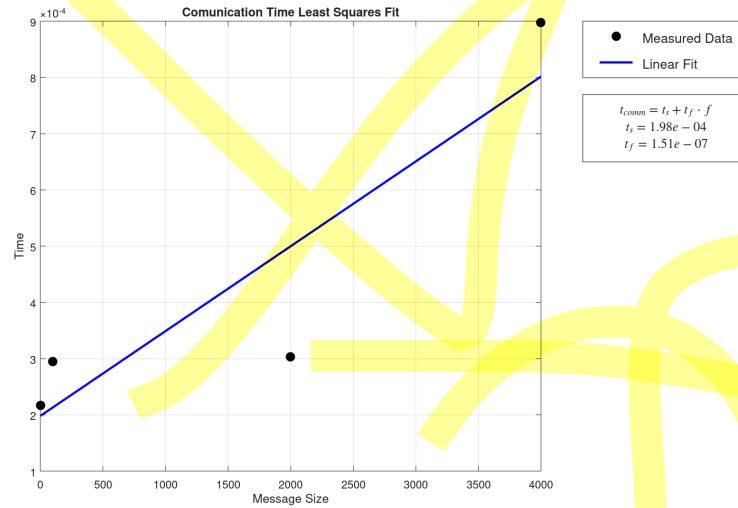


2 orders of magnitude. This shows that the algorithm is highly communication bound. There was also a clear increase in runtime between the larger sized lists. This makes sense as these would require either more send/recieve requests or building a larger send buffer for both algorithms. With increasing processor number, the runtime visually appears to be increasing linearly, reflecting that there is some overhead in the increased network size.

5 Least Squares fit for varied message size

A least squared fit was performed on the data provided in the assignment. The resulting values as well as the fitted parameters are shown in Figure 3. This was done using `matlab` and the code is shown in C.

Figure 3: Least squared fit for increasing runtime complexity



A Global arithmetic algorithm code

A.1 MPI Implementation of the global sum up and global series operations algorithms

```
#include <math.h>
#include <mpi.h>
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

/**
 * @brief Flips the bit in np at the position toFlip, starting with smallest
 * place value.
 *
 * @param np The number to flip a bit in.
 * @param bit The bit number to flip
 * @return The number np with the bit at position toFlip flipped
 */
int flip(int np, unsigned int bit) { return np ^ (1 << bit); }

/**
 * @brief Calculates the sum of every value in the network, returning it in
 * every processor.
 */
```

```

* @warning This function expects MPI_COMM_WORLD to be initialized.
* @warning This function expects size to be a power of 2.
*
* @note This function is designed to be used with a d-dimensional hypercube
* network of processors.
*
* @param rank The MPI rank of this processor for MPI_COMM_WORLD
* @param size The number of processors in MPI_COMM_WORLD
* @param a The number that is assigned to the current address.
*/
float globalSumUp(int rank, int size, float a) {
    int d = log2(size);
    float incomingVal;
    MPI_Request request = NULL;

    for (int i = 0; i < d; i++) {
        if (request != NULL) {
            MPI_Wait(&request, MPI_STATUS_IGNORE);
        }
        MPI_Isend(&a, 1, MPI_FLOAT, flip(rank, i), 0, MPI_COMM_WORLD, &request);

        // MPI_STATUS_IGNORE is used since each processor sends and receives the
        // same message.
        MPI_Recv(&incomingVal, 1, MPI_FLOAT, flip(rank, i), 0, MPI_COMM_WORLD,
            MPI_STATUS_IGNORE);

        a = a + incomingVal;
    }
    return a;
}

typedef enum { SUM, PRODUCT, MAX, MIN } Operation;

/**
* @brief Applies all the instructions in operations to every value a stored in
* each processor, distributing the result to all processors.
*
* @warning This function expects MPI_COMM_WORLD to be initialized.
* @warning This function expects size to be a power of 2.
* @warning results must be large enough to accept operationCount entries.
*
* @note This function is designed to be used with a d-dimensional hypercube
* network of processors.
*
* @param[out] results An array of return values in the same order as operations
*

```

```

* @param[in] rank The MPI rank of this processor for MPI_COMM_WORLD
* @param[in] size The number of processors in MPI_COMM_WORLD
* @param[in] a The number that is assigned to the current address.
* @param[in] operations A series of operations that should be applied to every
* number in the network.
* @param[in] operationCount The number of entries in operations
*/
void globalInstructionSet(float *results, int rank, int size, float a,
                        Operation operations[], int operationCount) {
    float receive[operationCount];
    MPI_Request request = NULL;

    // Initialize results buffer with a
    for (int op = 0; op < operationCount; op++) {
        results[op] = a;
    }

    int d = log2(size);
    for (int i = 0; i < d; i++) {
        if (request != NULL) {
            MPI_Wait(&request, MPI_STATUS_IGNORE);
        }
        MPI_Isend(results, operationCount, MPI_FLOAT, flip(rank, i), 0,
                 MPI_COMM_WORLD, &request);

        MPI_Recv(receive, operationCount, MPI_FLOAT, flip(rank, i), 0,
                 MPI_COMM_WORLD, MPI_STATUS_IGNORE);

        for (int op = 0; op < operationCount; op++) {
            switch (operations[op]) {
                case (SUM):
                    results[op] = receive[op] + results[op];
                    break;

                case (PRODUCT):
                    results[op] = receive[op] * results[op];
                    break;

                case (MAX):
                    results[op] = fmax(receive[op], results[op]);
                    break;

                case (MIN):
                    results[op] = fmin(receive[op], results[op]);
                    break;
            }
        }
    }
}

```

```

    }
}
}

```

A.2 Code used to profile the global sum up and global series operations algorithms

```

#include "globalArith.h"
#include <math.h>
#include <mpi.h>
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

long int getNanoseconds(struct timespec time) {
    return (time.tv_sec * 1e9) + time.tv_nsec;
}

void profileSum(double *t1, double *t2, int rank, int size) {
    srand(time(NULL) + rank);
    float number = (float)rand() / (float)RAND_MAX;

    struct timespec start, end;
    const int NIT = 50;
    float floatSendBuf, floatReceiveBuf;
    long int longReceiveBuf;

    *t1 = 0;
    *t2 = 0;

    floatSendBuf = number;
    for (int i = 0; i < NIT; i++) {
        MPI_Barrier(MPI_COMM_WORLD);
        clock_gettime(CLOCK_MONOTONIC, &start);
        MPI_Allreduce(&floatSendBuf, &floatReceiveBuf, 1, MPI_FLOAT, MPI_SUM,
                     MPI_COMM_WORLD);
        clock_gettime(CLOCK_MONOTONIC, &end);

        long int elapsed = getNanoseconds(end) - getNanoseconds(start);
        MPI_Allreduce(&elapsed, &longReceiveBuf, 1, MPI_LONG, MPI_MAX,
                     MPI_COMM_WORLD);
        *t1 += longReceiveBuf;
    }

    for (int i = 0; i < NIT; i++) {
        MPI_Barrier(MPI_COMM_WORLD);
    }
}

```

```

        clock_gettime(CLOCK_MONOTONIC, &start);
        globalSumUp(rank, size, number);
        clock_gettime(CLOCK_MONOTONIC, &end);

        long int elapsed = getNanoseconds(end) - getNanoseconds(start);
        MPI_Allreduce(&elapsed, &longReceiveBuf, 1, MPI_LONG, MPI_MAX,
                      MPI_COMM_WORLD);
        *t2 += longReceiveBuf;
    }
    *t1 = *t1 / NIT;
    *t2 = *t2 / NIT;
};

void profileSeries(double *t1, double *t2, int rank, int size) {
    srand(time(NULL) + rank);
    float number = (float)rand() / (float)RAND_MAX;
    struct timespec start, end;
    const int NIT = 50;

    float floatSendBuf, floatReceiveBuf;
    long int longReceiveBuf;

    *t1 = 0;
    *t2 = 0;

    floatSendBuf = number;
    for (int i = 0; i < NIT; i++) {
        MPI_Barrier(MPI_COMM_WORLD);
        clock_gettime(CLOCK_MONOTONIC, &start);
        MPI_Allreduce(&floatSendBuf, &floatReceiveBuf, 1, MPI_FLOAT, MPI_SUM,
                      MPI_COMM_WORLD);
        MPI_Allreduce(&floatSendBuf, &floatReceiveBuf, 1, MPI_FLOAT, MPI_PROD,
                      MPI_COMM_WORLD);
        MPI_Allreduce(&floatSendBuf, &floatReceiveBuf, 1, MPI_FLOAT, MPI_MIN,
                      MPI_COMM_WORLD);
        MPI_Allreduce(&floatSendBuf, &floatReceiveBuf, 1, MPI_FLOAT, MPI_MAX,
                      MPI_COMM_WORLD);
        MPI_Allreduce(&floatSendBuf, &floatReceiveBuf, 1, MPI_FLOAT, MPI_MAX,
                      MPI_COMM_WORLD);
        clock_gettime(CLOCK_MONOTONIC, &end);

        long int elapsed = getNanoseconds(end) - getNanoseconds(start);
        MPI_Allreduce(&elapsed, &longReceiveBuf, 1, MPI_LONG, MPI_MAX,
                      MPI_COMM_WORLD);
        *t1 += longReceiveBuf;
    }
}

```

```

int opcount = 5;
Operation ops[] = {SUM, PRODUCT, MIN, MAX, MAX};
float res[opcount];
for (int i = 0; i < NIT; i++) {
    MPI_Barrier(MPI_COMM_WORLD);
    clock_gettime(CLOCK_MONOTONIC, &start);
    globalInstructionSet(res, rank, size, number, ops, opcount);
    clock_gettime(CLOCK_MONOTONIC, &end);

    long int elapsed = getNanoseconds(end) - getNanoseconds(start);
    MPI_Allreduce(&elapsed, &longReceiveBuf, 1, MPI_LONG, MPI_MAX,
        MPI_COMM_WORLD);
    *t2 += longReceiveBuf;
}
*t1 = *t1 / NIT;
*t2 = *t2 / NIT;
};

int main(int argc, char **argv) {
    int rank, size;
    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    if (fmod(log2(size), 1.0) != 0) {
        printf("Number of processors should be a power of 2\n");
        MPI_Abort(MPI_COMM_WORLD, -1);
        return -1;
    }

    if (rank == 0 && size == 2) {
        FILE *results = fopen("./results.txt", "a");
        fprintf(results, "reduceSum,customSum,reduceSeries,customSeries\n");
        fclose(results);
    }

    double t1, t2;
    profileSum(&t1, &t2, rank, size);
    if (rank == 0) {
        FILE *results = fopen("./results.txt", "a");
        fprintf(results, "%lf,%lf,", t1, t2);
        fclose(results);
    }

    profileSeries(&t1, &t2, rank, size);

```

```

if (rank == 0) {
    FILE *results = fopen("./results.txt", "a");
    fprintf(results, "%lf,%lf\n", t1, t2);
    fclose(results);
}

MPI_Finalize();
return 0;

```

A.3 Matplotlib code used to plot results and run experiments

```

import subprocess
import pandas as pd
import matplotlib.pyplot as plt
from matplotlib.ticker import ScalarFormatter

def main():
    procSizes = [2 ** i for i in range(1, 5)]
    subprocess.run(["make", "clean"])
    for procs in procSizes:
        subprocess.run(["make", "run", f"ARGUMENTS={procs}"], check=True)

    data = pd.read_csv("./results.txt")

    fig, ax = plt.subplots()
    ax.set_xscale('log', base=2)
    formatter = ScalarFormatter()
    formatter.set_scientific(False)
    ax.xaxis.set_major_formatter(formatter)

    plt.grid(True)

    plt.plot(procSizes, data["customSum"],
             label="Global sum algorithm", marker="o", linestyle="-")
    plt.plot(procSizes, data["reduceSum"],
             label="MPI_AllReduce", marker="s", linestyle="--")
    plt.plot(procSizes, data["customSeries"],
             label="Global serial operation algorithm (5 operations)",
             marker="x", linestyle="-")
    plt.plot(procSizes, data["reduceSeries"],
             label="MPI_AllReduce series (5 operations)", marker="^",
             linestyle="--")
    plt.legend()

```

```

plt.xlabel("Number of Processors (log scale)", fontsize=12)
plt.ylabel("Execution Time (ns)", fontsize=12)
plt.title("Performance: Custom Implementation vs. MPI Reduction",
          fontsize=14)

plt.legend(frameon=True)

plt.savefig("globalArithProfile.pdf")

plt.cla()

```

```
main()
```

B Batched vs individual communication for a vector communication problem

B.1 Implementation of component wise and vectorized data transfer using MPI

```

#include <math.h>
#include <mpi.h>
#include <stdio.h>

typedef enum { PREV, NEXT } Direction;

/**
 * @brief Returns the rank of the neighbour in the cyclic linear array
 *
 * @param rank Rank of the current processor
 * @param size Number of processors in MPI_COMM_WORLD
 *
 * @returns The rank of the next processor in the cyclic linear array
 */
int getNeighbour(int rank, int size, Direction dir) {
    int neighbour;
    switch (dir) {
        case PREV:
            neighbour = rank - 1;
            if (neighbour == -1) {
                return size - 1;
            } else {
                return neighbour;
            }
    }
}

```

```

        break;
    case NEXT:
        neighbour = rank + 1;
        if (neighbour == size) {
            return 0;
        } else {
            return neighbour;
        }
    }
}

/**
 * @brief Passes the values in vec at locations a * vecSize + 1 for a =
 * {1,...,n-1} to the next vector in the cycle and receives the same values from
 * the previous neighbour. Passes each component in its own MPI call.
 *
 *
 * @note Designed to be used on a cyclic linear array
 * @warning Assumes that MPI_Init has already been called
 * @warning Assumes that vecSize is a square of a whole number.
 *
 * @param rank Rank of the current processor
 * @param size Number of processors in MPI_COMM_WORLD
 * @param vec Vector of values to be sent and recieved
 * @param vecSize Number of elements in vec
 *
 * @returns 0 on success or -1 on failure
 */
int passSubVectorByComponent(int rank, int size, float vec[], int vecSize) {
    int recBuf;
    MPI_Request req;

    int n = sqrt(vecSize);

    for (int i = 0; i < n; i++) {
        int idx = n * i + 1;

        if (MPI_Isend(vec + idx, 1, MPI_FLOAT, getNeighbour(rank, size, NEXT),
                     0, MPI_COMM_WORLD, &req) != MPI_SUCCESS) {
            return -1;
        }

        if (MPI_Recv(&recBuf, 1, MPI_FLOAT, getNeighbour(rank, size, PREV), 0,
                    MPI_COMM_WORLD, MPI_STATUS_IGNORE) != MPI_SUCCESS) {
            return -1;
        }
    }
}

```

```

        if (MPI_Wait(&req, MPI_STATUS_IGNORE) != MPI_SUCCESS) {
            return -1;
        }

        vec[idx] = recBuf;
    }
    return 0;
}

/**
 * @brief Passes the values in vec at locations a * vecSize + 1 for a =
 * {1,...,n-1} to the next vector in the cycle and receives the same values from
 * the previous neighbour. Passes vectors in one MPI call.
 *
 *
 * @note Designed to be used on a cyclic linear array
 * @warning Assumes that MPI_Init has already been called
 * @warning Assumes that vecSize is a square of a whole number.
 *
 * @param rank Rank of the current processor
 * @param size Number of processors in MPI_COMM_WORLD
 * @param vec Vector of values to be sent and recieved
 * @param vecSize Number of elements in vec
 *
 * @returns 0 on success or -1 on failure
 */
int passSubVectorGrouped(int rank, int size, float vec[], int vecSize) {

    int n = sqrt(vecSize);

    int recBuf[n];
    int sendBuf[n];

    MPI_Request req;

    for (int i = 0; i < n; i++) {
        int idx = i * n + 1;
        sendBuf[i] = vec[idx];
    }

    if (MPI_Isend(sendBuf, n, MPI_FLOAT, getNeighbour(rank, size, NEXT), 0,
        MPI_COMM_WORLD, &req) != MPI_SUCCESS) {
        return -1;
    }
}

```

```

    if (MPI_Recv(&recBuf, n, MPI_FLOAT, getNeighbour(rank, size, PREV), 0,
        MPI_COMM_WORLD, MPI_STATUS_IGNORE) != MPI_SUCCESS) {
        return -1;
    }

    if (MPI_Wait(&req, MPI_STATUS_IGNORE) != MPI_SUCCESS) {
        return -1;
    }

    for (int i = 0; i < n; i++) {
        int idx = i * n + 1;
        vec[idx] = recBuf[i];
    }

    return 0;
}

```

B.2 Code used to profile both the component wise and vectorized data transfer

```

#include "linearArray.h"
#include <mpi.h>
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

long int getNanoseconds(struct timespec time) {
    return (time.tv_sec * 1e9) + time.tv_nsec;
}

int main(int argc, char **argv) {
    int rank, size, result;
    struct timespec start, end;
    long int t1[5], t2[5];

    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    for (int i = 0; i < 5; i++) {
        int n = 1000 * (1 << i);
        int vecSize = (int)pow(n, 2);

        float *vec = malloc(vecSize * sizeof(float));

        MPI_Barrier(MPI_COMM_WORLD);
    }
}

```

```

        clock_gettime(CLOCK_MONOTONIC, &start);
        passSubVectorByComponent(rank, size, vec, vecSize);
        clock_gettime(CLOCK_MONOTONIC, &end);
        t1[i] = getNanoseconds(end) - getNanoseconds(start);

        if (result != 0) {
            MPI_Abort(MPI_COMM_WORLD, -1);
        }

        MPI_Barrier(MPI_COMM_WORLD);
        clock_gettime(CLOCK_MONOTONIC, &start);
        result = passSubVectorGrouped(rank, size, vec, vecSize);
        clock_gettime(CLOCK_MONOTONIC, &end);
        t2[i] = getNanoseconds(end) - getNanoseconds(start);
        if (result != 0) {
            MPI_Abort(MPI_COMM_WORLD, -1);
        }

        free(vec);
    }

    if (rank == 0) {
        FILE *resultFile = fopen("./results.txt", "a");
        if (size == 2) {
            fprintf(resultFile, "size,separate,grouped\n");
        }
        for (int i = 0; i < 5; i++) {
            fprintf(resultFile, "%d,%ld,%ld\n", 1000 * (1 << i), t1[i], t2[i]);
        }
        fclose(resultFile);
    }
    MPI_Finalize();
    return 0;
}

```

B.3 Code used to plot values and run simulations for the batched vs individual comparison

```

import subprocess
import pandas as pd
import matplotlib.pyplot as plt
from matplotlib.ticker import ScalarFormatter

def main():
    procSizes = [2 ** i for i in range(1, 5)]

```

```

# subprocess.run(["make", "clean"])
# for procs in procSizes:
#     subprocess.run(["make", "run", f"ARGUMENTS={procs}"], check=True)
#
fig, ax = plt.subplots()

ax.set_xscale('log', base=2)
ax.set_yscale('log', base=10)

formatter = ScalarFormatter()
formatter.set_scientific(False)
ax.xaxis.set_major_formatter(formatter)

data = pd.read_csv("./results.txt")
plt.plot(procSizes, (data[data['size'] == 4000]
    ["separate"]), linestyle="-", marker="^",
    label="Individually sending components (n =4000)")
plt.plot(procSizes, (data[data['size'] == 8000]
    ["separate"]), linestyle="-", marker="o",
    label="Individually sending components (n=8000)")
plt.plot(procSizes, (data[data['size'] == 16000]
    ["separate"]), linestyle="-", marker="x",
    label="Individually sending components (n=16000)")
plt.plot(procSizes, (data[data['size'] == 4000]
    ["grouped"]), linestyle="--", marker="^",
    label="Batch sending components (n=4000)")

plt.plot(procSizes, (data[data['size'] == 8000]
    ["grouped"]), linestyle="--", marker="o",
    label="Batch sending components (n=8000)")
plt.plot(procSizes, (data[data['size'] == 16000]
    ["grouped"]), linestyle="--", marker="x",
    label="Batch sending components (n=16000)")

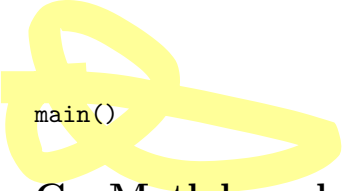
plt.grid(True)
fig.legend(loc='upper center', bbox_to_anchor=(0.5, 0))
plt.xlabel("Number of Processors (log scale)", fontsize=12)
plt.ylabel("Execution Time (ns) (log scale)", fontsize=12)
plt.title("Performance Comparison: Individual vs Batched communication",
    fontsize=14)

plt.subplots_adjust(bottom=0.2)

plt.savefig("linearArrayProfile.pdf", bbox_inches='tight')

plt.cla()

```



```
main()
```

C Matlab code used to calculate least squares fit for provided data

```
b = [2.17e-4; 2.95e-4; 3.03e-4; 8.98e-4];
sizes = [1;100;2000;4000];
A = [ones(length(sizes), 1), sizes];

x = A\b;
t_s = x(1);
t_f = x(2);

figure(1);
clf;

ax = axes('Position', [0.1, 0.1, 0.6, 0.8], 'NextPlot', 'add');
vals = plot(sizes, b, 'ko', 'MarkerFaceColor', 'k', 'MarkerSize', 8);
fit = plot(sizes, A*x, 'b-', 'LineWidth', 2);

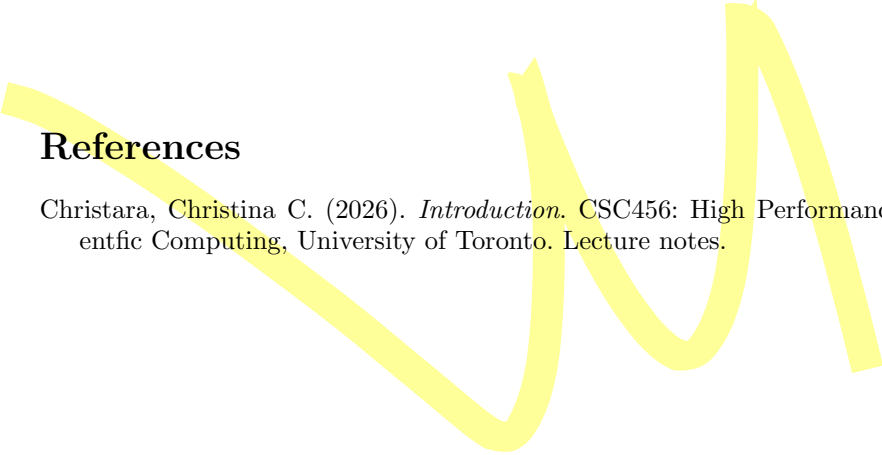
xlabel('Message Size');
ylabel('Time');
title('Communication Time Least Squares Fit');
grid on;
box on;

lgd = legend([vals, fit], {' Measured Data', ' Linear Fit'}, ...
    'Position', [0.75, 0.8, 0.2, 0.1], ...
    'FontSize', 12, ...
    'Box', 'on', ...
    'EdgeColor', 'k');

str = {
    '$t_{comm} = t_s + t_f \cdot f$',
    sprintf('$t_s = %.2e$', t_s),
    sprintf('$t_f = %.2e$', t_f)
};

annotation('textbox', [0.75, 0.65, 0.2, 0.12], ...
    'String', str, ...
    'Interpreter', 'latex', ...
    'FontSize', 12, ...
```

```
'BackgroundColor', 'w', ...  
'EdgeColor', 'k', ...  
'HorizontalAlignment', 'center', ...  
'VerticalAlignment', 'middle');  
  
hold off;
```



References

Christara, Christina C. (2026). *Introduction*. CSC456: High Performance Scientific Computing, University of Toronto. Lecture notes.